Developing Intuition on Building Blocks

Part 3 of 12 · Hands-On Project Companion

Event-Driven eCommerce Platform

with Apache Kafka, Spring Boot, Google Cloud Pub/Sub and PostgreSQL

Stack	Spring Boot · Apache Kafka · Google Cloud Pub/Sub · PostgreSQL · Docker Compose · Confluent Control Cent
Concepts covered	Key-based partitioning, ISR replication, consumer groups, idempotent producer, transactional read-process-write (EOS), log compaction, consumer lag monitoring, Kafka + Pub/Sub bridge pattern
Prerequisites	Main article — Event Streaming section; Deep Dives 1, 2, 3 recommended
GitHub	github.com/DhruvJawalkar — see repository for full source and phase instructions

Contents

1. The Project
2. Why This Project — Concept Coverage Map
3. Architecture Overview
4. Kafka Topic and Database Design Decisions
5. Message Schemas
6. Order Service — Producer Implementation
7. Consumer Implementations
8. Operational Observability
9. What to Try
10. Extending the Project

1. The Project

An event-driven order processing platform across four independent services, connected by Apache Kafka as the backbone. The Notification Service introduced in the Pub-Sub section becomes a full component here — the Notification Bridge consumes from Kafka and fans out to Google Cloud Pub/Sub, showing how the two messaging models complement each other in a real system.

The platform is deliberately scoped to make every Kafka concept from the main article and deep dives observable in a running system: key-based partitioning on orderId, ISR replication across a 3-node cluster, three independent consumer groups, an idempotent producer on the Order Service, and the transactional read-process-write pattern on the Payment Service for exactly-once semantics. Consumer lag across all three groups is visible in real time via Confluent Control Center.

The bridge pattern — the Notification Bridge Service consuming from Kafka and publishing to GCP Pub/Sub — demonstrates a real architectural decision: Kafka for internal event streaming between services, Pub/Sub for external fan-out to notification channels. This pattern appears frequently in systems that use Kafka internally but need to integrate with managed cloud services for outbound communication.

What It Deliberately Excludes

- Authentication / JWT — hardcoded service identities to keep focus on Kafka topology
- Horizontal scaling / multiple instances — covered conceptually, not deployed
- Schema Registry / Avro — JSON payloads for simplicity; Schema Registry is the natural next step
- Full frontend UI — REST endpoints only; observable via curl, Postman, or Control Center

2. Why This Project — Concept Coverage Map

Every design decision in this project maps to a concept from the article and deep dives. The table below makes those connections explicit.

What You Build	Concept Solidified	Practical Transfer
12-partition orders topic with key=orderId	Key-based partitioning — per-key ordering across partitions	Any system where entity-level event ordering matters
3-node Kafka cluster, replication.factor=3, min.insync.replicas=2	ISR replication and ACK levels (Deep Dive 1)	Production Kafka cluster configuration for business-critical topics
Three independent consumer groups on the orders topic	Consumer groups — fan-out without data copying	Multiple services independently consuming the same event stream
enable.idempotence=true on Order Service producer	Idempotent producer — PID/sequence deduplication (Deep Dive 3)	Every production Kafka producer handling business events
Payment Service transactional read-process-write	Exactly-once semantics — beginTransaction / sendOffsetsToTransaction / commitTransaction (Deep Dive 3)	Financial pipelines, inventory deductions, irreversible side effects
isolation.level=read_committed on Payment consumer	Consumer-side EOS guarantee — no dirty reads from aborted transactions	Required on all consumers reading from transactional topics
payments topic with cleanup.policy=compact	Log compaction — latest value per key retained indefinitely (Deep Dive 1)	Entity state topics: current payment status, account balance, inventory level
Notification Bridge → GCP Pub/Sub	Kafka + Pub/Sub bridge pattern — internal streaming + external fan-out	Integrating internal Kafka backbone with managed cloud notification services
Consumer lag monitoring across all three groups	Consumer lag as the primary operational throughput signal (Deep Dive 2)	Production Kafka observability — knowing when consumers are falling behind
Confluent Control Center + kafka-consumer-groups.sh	Operational observability — ISR status, partition assignment, throughput	Production Kafka cluster operations and incident diagnosis

3. Architecture Overview

The platform has four Spring Boot services, a 3-node Kafka cluster, Google Cloud Pub/Sub for notification fan-out, two PostgreSQL databases, and Confluent Control Center for observability — all started with a single `docker-compose up`.

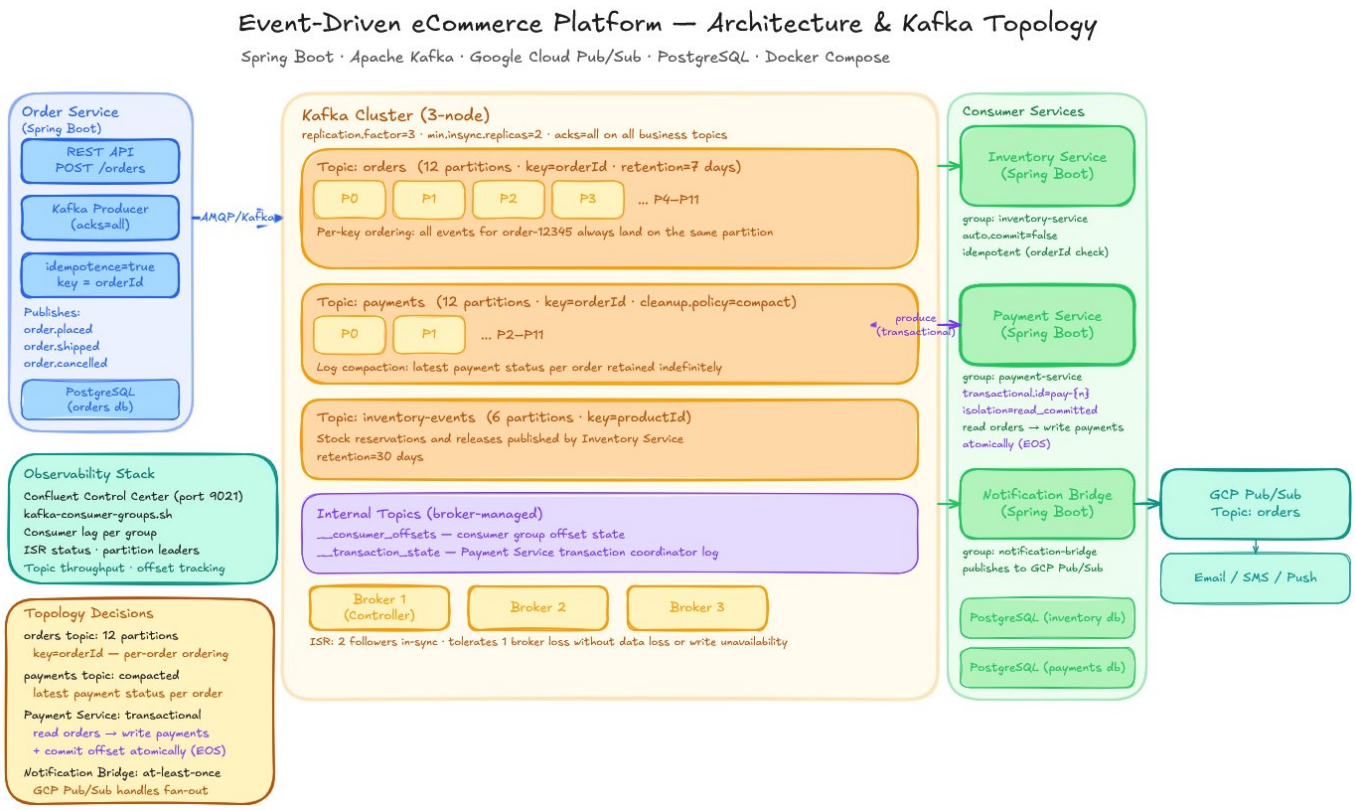

Figure 1 — Event-Driven eCommerce Platform Architecture & Kafka Topology [https://link.excalidraw.com/l/5sVhqUivTXa/6XmscHWLlgc]

Service	Role	Key Behaviours
Order Service (Spring Boot)	System entry point — REST API + Kafka producer	Receives POST /orders. Persists to orders db. Publishes order.placed, order.shipped, order.cancelled to orders topic with acks=all, enable.idempotence=true, key=orderId.
Inventory Service (Spring Boot)	Stock reservation and release	Consumer group: inventory-service. Reads orders topic. Reserves stock on order.placed, releases on order.cancelled. Manual ack, idempotent (orderId check before write). Persists to inventory db.
Payment Service (Spring Boot)	Payment processing — exactly-once	Consumer group: payment-service. Transactional read-process-write: reads orders topic, processes payment, writes to payments topic, commits offset atomically. transactional.id, isolation.level=read_committed.

Notification Bridge (Spring Boot)	Kafka → GCP Pub/Sub bridge	Consumer group: notification-bridge. Reads orders topic. Publishes to GCP Pub/Sub topic. At-least-once — GCP Pub/Sub handles fan-out to email/SMS/push via the Notification Service from the Pub/Sub section.

docker-compose.yml — key services

```
services:
zookeeper:
image: confluentinc/cp-zookeeper:7.5.0
environment: { ZOOKEEPER_CLIENT_PORT: 2181 }

kafka-1:
image: confluentinc/cp-kafka:7.5.0
environment:
KAFKA_BROKER_ID: 1
KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka-1:9092
KAFKA_DEFAULT_REPLICATION_FACTOR: 3
KAFKA_MIN_INSYNC_REPLICAS: 2
kafka-2: # same pattern, BROKER_ID: 2
kafka-3: # same pattern, BROKER_ID: 3

control-center:
image: confluentinc/cp-enterprise-control-center:7.5.0
ports: ['9021:9021']
depends_on: [kafka-1, kafka-2, kafka-3]

postgres:
image: postgres:15
environment: { POSTGRES_DB: ecommercedb }

order-service:
build: ./order-service
ports: ['8080:8080']
depends_on: [kafka-1, postgres]

inventory-service:
build: ./inventory-service
depends_on: [kafka-1, postgres]

payment-service:
build: ./payment-service
depends_on: [kafka-1, postgres]

notification-bridge:
build: ./notification-bridge
depends_on: [kafka-1]
```

4. Kafka Topic and Database Design Decisions

Every topic configuration and database choice in this project was made for a specific reason. Understanding these decisions is more valuable than the code that implements them.

Topic Design

Topic	Config	Why This Choice
orders	12 partitions, key=orderId, retention=7 days, acks=all	12 partitions allows up to 12 parallel consumers per group. key=orderId routes all events for the same order to the same partition, guaranteeing per-order ordering. 7-day retention allows any consumer to replay a week of history.
payments	12 partitions, key=orderId, cleanup.policy=compact	Compacted because downstream consumers (reporting, dispute resolution) need current payment state, not raw history. Log compaction retains the latest payment status per orderId indefinitely — new consumers bootstrap current state without replaying years of events.
inventory-events	6 partitions, key=productId, retention=30 days	6 partitions because inventory processing is less parallel than order processing. key=productId ensures all events for the same product are ordered. 30-day retention for stock reconciliation queries.

Why PostgreSQL Alongside Kafka

This is the question the topology raises immediately: if all events are in Kafka, why persist to PostgreSQL at all? The answer is that Kafka and PostgreSQL solve different problems — they are complementary, not redundant.

Capability	Kafka	PostgreSQL	Winner
Sequential event delivery	Yes — ordered by partition offset	No — rows have no inherent order	Kafka
Ad-hoc queries ('all orders by user_alice this month')	No — requires full topic scan	Yes — indexed SQL queries	PostgreSQL
Idempotency check before write (atomic with business operation)	No — cannot do check-then-write transactionally	Yes — CHECK + INSERT in one transaction	PostgreSQL
Current state of an entity	Only with log compaction + full replay	Yes — single row lookup	PostgreSQL

Historical replay (last 7 days of orders)	Yes — seek to any offset	Only if explicitly stored	Kafka
Legal / financial retention (7+ years)	No — retention is bounded by cost and time	Yes — with appropriate backup policies	PostgreSQL

The mental model: Kafka is the nervous system — events flow through it to decouple and coordinate services. PostgreSQL is the memory — each service maintains the queryable state it needs for its own business logic and API responses. Removing PostgreSQL and using only Kafka would require full-topic-scan queries, no atomic idempotency checks, and bounded history. Each payment record also has legal retention requirements well beyond Kafka's retention window — the payments database is the durable ledger; the payments topic is the distribution mechanism.

5. Message Schemas

All messages are JSON-serialised. The Order Service sets a `messageId` (UUID) on every published event — used for idempotency checks in Inventory and Payment consumers.

// OrderEvent.java + JSON wire format

```
public record OrderEvent(
String messageId, // UUID — set by producer
String eventType, // ORDER_PLACED | ORDER_SHIPPED | ORDER_CANCELLED
String orderId, // Kafka partition key
String customerId,
List<OrderItem> items,
BigDecimal totalAmount,
String currency, // ISO 4217
long timestamp
) {}

// Wire format
{
"messageId": "550e8400-e29b-41d4-a716-446655440000",
"eventType": "ORDER_PLACED",
"orderId": "order-12345",
"customerId": "customer-alice",
"items": [{"sku": "SKU-999", "qty": 2, "price": 29.99}],
"totalAmount": 59.98,
"currency": "USD",
"timestamp": 1714000000000
}
```

// PaymentEvent.java — written to payments topic (compacted)

```
public record PaymentEvent(
String messageId,
String orderId, // Kafka partition key + compaction key
String status, // PENDING | AUTHORISED | CAPTURED | FAILED | REFUNDED
BigDecimal amount,
String paymentMethod,
long timestamp
) {}
// Compaction key = orderId: latest status per order retained indefinitely
```

6. Order Service — Producer Implementation

The Order Service is the sole producer of order events. It uses an idempotent producer with `acks=all` — any network retry is safely deduplicated by the broker via PID + sequence number tracking.

// KafkaProducerConfig.java

```
@Configuration
public class KafkaProducerConfig {

@Bean
public ProducerFactory<String, OrderEvent> producerFactory() {
Map<String, Object> config = new HashMap<>();
config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"kafka-1:9092,kafka-2:9092,kafka-3:9092");
config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);

// Durability
config.put(ProducerConfig.ACKS_CONFIG, "all");
config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
config.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);

// Throughput
config.put(ProducerConfig.BATCH_SIZE_CONFIG, 16384);
config.put(ProducerConfig.LINGER_MS_CONFIG, 5);
config.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");
return new DefaultKafkaProducerFactory<>(config);
}

@Bean
public KafkaTemplate<String, OrderEvent> kafkaTemplate(
ProducerFactory<String, OrderEvent> pf) {
return new KafkaTemplate<>(pf);
}
}
```

// OrderService.java — publish with orderId as key

```
@Service
public class OrderService {

private final KafkaTemplate<String, OrderEvent> kafka;
private final OrderRepository repository;

public Order placeOrder(PlaceOrderRequest request) {
Order order = Order.create(request);
repository.save(order); // Persist to PostgreSQL first

OrderEvent event = OrderEvent.builder()
.messageId(UUID.randomUUID().toString()) // Set by producer
```

```
.eventType("ORDER_PLACED")
.orderId(order.getId()) // Partition key
.customerId(request.customerId())
.items(request.items())
.totalAmount(order.getTotalAmount())
.timestamp(System.currentTimeMillis())
.build();

// key=orderId: all events for this order land on same partition
kafka.send("orders", order.getId(), event)
.whenComplete((result, ex) -> {
if (ex != null)
log.error("Failed to publish order event: {}", ex.getMessage());
else
log.info("Published to partition={} offset={}",
result.getRecordMetadata().partition(),
result.getRecordMetadata().offset());
});
return order;
}
}
```

7. Consumer Implementations

Inventory Service — At-Least-Once with Idempotency

// InventoryConsumer.java

```
@Component
public class InventoryConsumer {

private final InventoryRepository repository;

@KafkaListener(
topics = "orders",
groupId = "inventory-service",
containerFactory = "inventoryContainerFactory" // prefetch=50, manual ack
)
public void handleOrder(OrderEvent event,
@Header(KafkaHeaders.OFFSET) long offset,
Acknowledgment ack) {
try {
if (repository.existsByMessageId(event.messageId())) {
ack.acknowledge(); // Duplicate — safe to ack
return;
}
switch (event.eventType()) {
case "ORDER_PLACED" -> reserveStock(event);
case "ORDER_CANCELLED" -> releaseStock(event);
}
ack.acknowledge();
} catch (InsufficientStockException e) {
log.error("Stock reservation failed for {}: {}",
event.orderId(), e.getMessage());
ack.acknowledge(); // Ack — publish compensating event instead
} catch (Exception e) {
log.error("Inventory consumer error: {}", e.getMessage());
// Do not ack — will be redelivered
}
}
}
```

Payment Service — Exactly-Once Transactional

// PaymentConsumer.java — transactional read-process-write

```
@Component
public class PaymentConsumer {

private final KafkaTemplate<String, PaymentEvent> kafkaTemplate;
private final PaymentRepository repository;
```

```
@KafkaListener(
topics = "orders",
groupId = "payment-service",
containerFactory = "transactionalContainerFactory"
)
@Transactional("kafkaTransactionManager")
public void handleOrder(OrderEvent event,
ConsumerRecord<String, OrderEvent> record) {
if (!"ORDER_PLACED".equals(event.eventType())) return;

// Process payment
PaymentResult result = paymentGateway.charge(
event.customerId(), event.totalAmount(), event.currency());

PaymentEvent paymentEvent = PaymentEvent.builder()
.messageId(UUID.randomUUID().toString())
.orderId(event.orderId())
.status(result.success() ? "CAPTURED" : "FAILED")
.amount(event.totalAmount())
.timestamp(System.currentTimeMillis())
.build();

// Write to payments topic + commit offset atomically
// If either fails, both are rolled back — consumer re-reads on next poll
kafkaTemplate.send("payments", event.orderId(), paymentEvent);
repository.save(Payment.from(paymentEvent));
// Offset commit handled by @Transactional — sendOffsetsToTransaction
}
}
```

Why the Payment Service uses EOS but Inventory does not: Inventory stock reservation is naturally idempotent — reserving the same orderId twice is safely detected by the messageId check. Payment capture is not idempotent — charging a customer twice is a real business harm. The transactional API ensures the payment event is written to the payments topic and the offset committed atomically. If payment processing fails, the transaction aborts and the consumer re-reads the order event safely.

Notification Bridge — Kafka → GCP Pub/Sub

// NotificationBridgeConsumer.java

```
@Component
public class NotificationBridgeConsumer {

private final Publisher gcpPublisher; // GCP Pub/Sub publisher

@KafkaListener(topics = "orders", groupId = "notification-bridge")
public void handleOrder(OrderEvent event, Acknowledgment ack) {
// Only bridge ORDER_PLACED events to notification channel
if (!"ORDER_PLACED".equals(event.eventType())) {
```

```
ack.acknowledge();
return;
}
try {
PubsubMessage message = PubsubMessage.newBuilder()
.setData(ByteString.copyFromUtf8(toJson(event)))
.putAttributes("eventType", event.eventType())
.putAttributes("orderId", event.orderId())
.build();
gcpPublisher.publish(message).get(); // Sync for at-least-once
ack.acknowledge();
} catch (Exception e) {
log.error("Bridge publish failed for {}: {}",
event.orderId(), e.getMessage());
// Do not ack — Kafka will redeliver
}
}
}
// GCP Pub/Sub then fans out to email, SMS, push via the
// Notification Service subscriptions from the Pub-Sub section.
```

8. Operational Observability

Confluent Control Center (http://localhost:9021) starts automatically via Docker Compose and provides a full cluster view without any additional setup.

What to Watch	Where	What It Tells You
Consumer lag per group	Control Center → Consumer Groups → Lag	Whether each service (inventory, payment, notification-bridge) is keeping up with the Order Service publish rate. Zero = healthy. Growing = consumer slower than producer.
Topic throughput	Control Center → Topics → orders → Throughput	Messages per second in and out. Confirm all three consumer groups are pulling at the expected rate.
ISR status	Control Center → Topics → orders → Partitions	Number of in-sync replicas per partition. Should always be 3 (or at minimum 2). ISR < 2 = durability at risk.
Partition assignment	Control Center → Consumer Groups → inventory-service → Partition Assignment	Which consumer instance owns which partition. Add a second inventory-service instance and observe rebalance.
Transaction state	kafka-consumer-groups.sh --group payment-service --describe	Committed offsets for the payment-service group. Verify offsets advance only after transaction commit.
__transaction_state topic	Control Center → Topics → __transaction_state	Broker-managed log of open and committed transactions. Observe entries during Payment Service processing.

9. What to Try

Open-ended experiments — observe system behaviour under conditions you rarely encounter in development.

Stop the Inventory Service mid-flow

Send 20 orders. Stop inventory-service. Watch consumer lag grow in Control Center. Restart it and observe the drain. Verify all stock reservations were made exactly once.

Kill a Kafka broker

Stop kafka-2. Watch ISR drop to 2 in Control Center. Verify the Order Service continues publishing (min.insync.replicas=2 still satisfied). Restart kafka-2 and watch it rejoin the ISR.

Force a transaction abort

Introduce a simulated payment gateway failure for orders over $1000. Send a large order. Observe the Payment Service transaction abort in logs. Verify the offset was NOT committed and the order is re-read on the next poll.

Observe consumer group rebalance

Start a second inventory-service instance. Watch partition assignments change in Control Center. Some partitions move to the new instance. Observe lag drop as two instances share the load.

Read from the compacted payments topic

Send 5 payment events for the same orderId with different statuses (PENDING → AUTHORISED → CAPTURED). After compaction runs, seek to the beginning of the payments topic. Only the CAPTURED event should remain for that orderId.

Observe the Kafka → Pub/Sub bridge latency

Send an order and watch it appear in GCP Pub/Sub (via the Notification Service from the Pub-Sub section). Measure end-to-end time from Kafka publish to notification delivery.

Test consumer lag alerting

Artificially slow down the Payment Service (add a Thread.sleep). Watch consumer lag grow. When lag exceeds your alert threshold, verify the alert fires.

10. Extending the Project

Extension	What It Adds	Concepts It Exercises
Schema Registry + Avro	Replace JSON with Avro schemas registered in Confluent Schema Registry. Add a new field to OrderEvent and observe BACKWARD compatibility enforcement at publish time.	Schema evolution from Pub-Sub Deep Dive 2 — applied to Kafka
Outbox Pattern on Order Service	Replace direct Kafka publish with a transactional outbox table + CDC (Debezium) to guarantee exactly-once between PostgreSQL write and Kafka publish.	The Outbox Pattern from the EDA section of the article
Multiple Payment Service instances	Run 3 payment-service instances. Observe transactional.id fencing when one instance fails and another takes over the same partitions.	Producer fencing and exactly-once under failure
Prometheus + Grafana consumer lag dashboard	Expose Kafka metrics via JMX Exporter. Build a Grafana dashboard showing lag per consumer group, topic throughput, and ISR status.	Production observability — the Hidden Costs section of the article
Add a Search Service	New consumer group reads orders topic and indexes into Elasticsearch. Demonstrates zero-cost fan-out — no changes to any existing service needed.	Consumer group isolation — adding consumers without touching producers
KRaft mode (remove ZooKeeper)	Switch the Kafka cluster to KRaft mode. Observe the simplified operational topology.	Modern Kafka operations — ZooKeeper dependency removal

Companion to: **Developing Intuition on Building Blocks — Part 3** · Message Queues, Publish-Subscribe Systems, Event Streaming Systems
Hands-On Project 2 of 2 · Full source: github.com/DhruvJawalkar